IERG4320/IEMS5726: Data Science in Practice

Lecture 4: Data Representation II

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

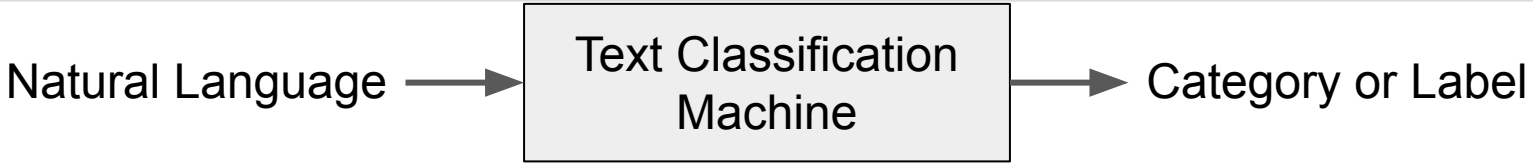
香港中文大學
The Chinese University of Hong Kong

Outline

1. Dealing with Natural Language
   - Classic Natural Language Processing
   - Word Embedding
2. Dealing with Image Data
   - Image Descriptors
   - Image Tensors
3. Dealing with Audio Data
   - Classic Audio Signal Processing
   - Mel Spectrogram and Masking

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

Text Classification



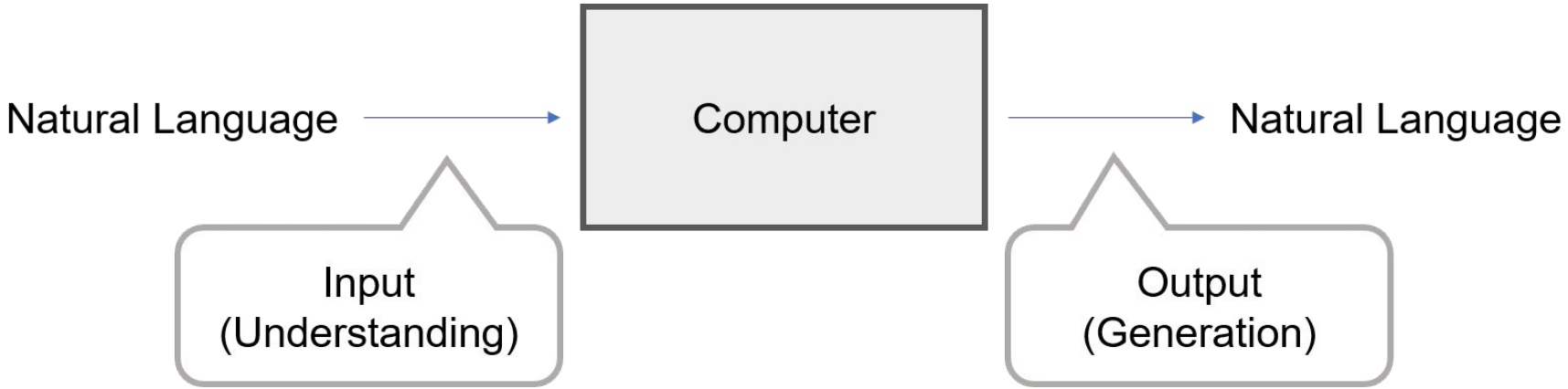
- Common applications:
  - Spam detection, sentiment analysis, topic labeling, intent detection, … etc
- Text data is a more difficult problem compared to category:
  - Data is highly **unstructured** (not in tabular format)
  - Data can be very **noisy** (spelling mistakes, typos)
  - Input can be very short (e.g. "Nice restaurant!"), and can also be very long (e.g. a detailed report of an event on a newspaper)
  - Unseen data always contain **unseen features** (new words, phrases, symbols or codes)
  - Many new features might come up over time (e.g. new jargons, new words)
  - Things can become very different across **different languages**

Features and Representation for Natural Language

- Since texts are **not structured** input, we need some ways to convert a text into a **vector representation** (feature vectors)
- What would be the features X of a text document?
  - Individual **words** / **phrases** / **sentences**
- Can also consider:
  - **Length** of the text
  - Existence of special **entities** in the text (e.g. person names, organizations, countries, etc.)
  - Source (e.g. which newspaper it is from)
  - **Date**/**time** of creation, publication, modification, etc.
  - … etc
- Do these numeric information represent the actual text content?

Natural Language Processing (NLP)

- NLP involves making the computer **understand** natural language input and **generate** natural language output
- It is a field that involves computer science, linguistics, artificial intelligence, human-computer interface, etc.



Examples: Real-time Speech Translation

1. Real-time Skype Translator by Microsoft Research
   https://www.youtube.com/watch?v=JrlTzS7Fk6o
2. AI Powered Machine Translation
   https://www.youtube.com/watch?v=GYiU3hyMMxs
3. Skype Translator: Speak Chinese like a local
   https://www.youtube.com/watch?v=mWTySUGXR2k

Classic Natural Language Processing

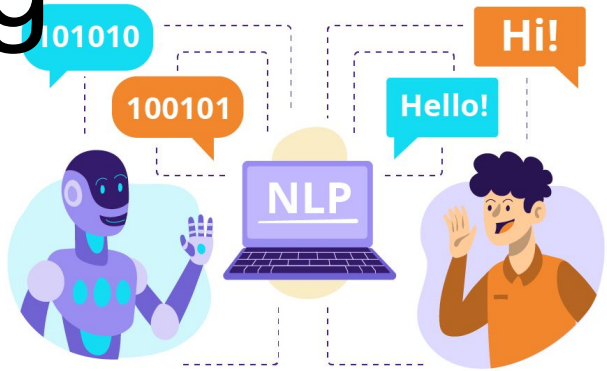


Image credit: Revolveai

How is NLP Done?

- Natural languages are usually **unstructured**, in which we find **alphabets**, **symbols**, **numbers and codes**, and even **emojis**.
- We need to apply different **preprocessing** algorithms before we can use the data for analysis and machine learning
- Common preprocessing tasks:
  - Tokenization
  - Stemming
  - Term Weighting
  - Parsing
  - Part-of-Speech (POS) Tagging
  - Pattern extraction using regular expressions

This is not the full list, different approaches may set different tasks. Som more examples: https://realpython.com/nltk-nlp-python/

Tokenization

- **Token** is a unit in natural language processing
- Usually it is a word in English (or other languages using Latin alphabets), separated by space
- **Tokenization** = breaking up the raw text into words (or other meaningful units)
- Problems:
  - How do we treat punctuations?
  - John’s book (John + ‘s + book ?)
  - Doesn't (does + not, or does + n’t ?)
  - Hyphenated words: so-called, high-risk, anti-social

Chinese (Asian Languages) Segmentation

- There are **no spaces** between words/phrases in Chinese and other **Asian languages** such as Japanese and Korean

超強颱風「山竹」目前集結太平洋地區，料明日登陸菲律賓，周日再吹襲香港。
美國氣象學家警告，「山竹」威力相等於 5級颶風，比吹襲美東颶風「佛羅倫斯」
更強。菲律賓政府嚴陣以待，並已疏散 120萬名沿岸居民。

大阪府北部地震や台風21号など度重なる災害を受け、京都市は14日、
各省庁への要望活動を始めた。停電の早期解消に向けた関西電力の指導や、
二条城などの文化財の復旧を支援する制度の拡充を国に求めた

스마트폰과 4세대(4G) 롱텀에볼루션(LTE)으로 재편된 휴대전화 시장에
2세대(2G)폰이 2년 만에 나온다. 삼성전자는 이달 중 폴더폰 '와이즈2 2G(모델명
SHC-Z160S)'를 SK텔레콤을 통해 출시한다. 국내 휴대전화 시장에 2G폰이 출시되는
것은 2011년 LG전자

Tokenizing in Python

- A commonly used tokenizer in English is the one provided by the Natural Language ToolKit (NLTK)
- Example:

```python
# installation under Anaconda Powershell Prompt: conda install -c anaconda nltk
import nltk
sentence = "Antoni Gaudí was a Spanish architect from Catalonia."
result = nltk.word_tokenize(sentence)
print(result)

sentence = "Every morning I wake up at about seven o'clock."
result = nltk.word_tokenize(sentence)
print(result)
```

```
['Antoni', 'Gaudí', 'was', 'a', 'Spanish', 'architect', 'from', 'Catalonia', '.']
['Every', 'morning', 'I', 'wake', 'up', 'at', 'about', 'seven', "o'clock", '.']
```

Tokenizing in Python

- In Chinese, a commonly used open source package is called jieba
- Example:

```python
# installation under Anaconda Powershell Prompt: conda install -c conda-forge jieba
import jieba
sentence = "超強颱風「山竹」目前集結太平洋地區，料明日登陸菲律賓，周日再吹襲香港。 "
tokens = list(jieba.cut(sentence))
print(tokens)
```

```
Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\Username\AppData\Local\Temp\jieba.cache
Loading model cost 0.567 seconds.
Prefix dict has been built successfully.
['超強', '颱', '風', '「', '山竹', '」', '目前', '集結', '太平洋', '地區', '，', '料', '明日', '
登陸菲律賓', '，', '周日', '再吹襲', '香港', '。']
```

Normalization

- A related issue: words in uppercase or lowercase
- E.g. Usually we do not want to treat house, House and HOUSE differently
- Normally, we convert all words into lower cases (lowercasing) (Problem?)
- Truecasing: try to preserve uppercase in entity names, in order to distinguish between something like `Mr. Brown` and `brown colour`.

Stemming

- A word may appear in different forms, consider:
  - cat, cats / bus, buses
  - run, running, runs, ran
  - fun, funny / beautiful, beautifully
- **Stemming** is the action of reducing words to its **stem** or **root**
  - cat, cats → cat
  - run, running, runs → run
- Many different ways to do this:
  - Lookup table
  - Rule-based (Suffix stripping)
  - Stochastic methods (machine learning)

Stemming

- The widely used stemming method used is the Porter Stemmer, invented by Martin F. Porter in 1980.
- Available in many different programming languages (e.g. C, C++, Python, Java, etc.)
- Javascript demo on a GitHub project.

```python
from nltk.stem.porter import PorterStemmer
stemmer = PorterStemmer()
a, b, c = stemmer.stem("running"), stemmer.stem("run"), stemmer.stem("runs")
print(a, b, c)

a, b = stemmer.stem("beauty"), stemmer.stem("beautiful")
print(a, b)
```

```
run run run
beauti beauti
```

Parts of Speech (POS)

- Words have different **roles** in a sentence:
  - **nouns** (e.g. house, car, people)
  - **verbs** (e.g. run, walk, pay, eat)
  - **adjectives** (e.g. beautiful, quick)
- Roughly, we can divide words into two broad categories:
  - Content words (e.g. nouns, verbs)
  - Function words (e.g. prepositions)
- **Content words** are also called **open-class** words (not a finite set of words, word can be created or become obsolete)
- **Function words** are called **close-class** words, because usually, they do not change over a long period of time.

Parts of Speech (POS)

Figure 2.5 Part-of-speech (POS) tags of Penn tree bank.

- **Nouns** refer to abstract or real objects in the word. Nouns (singular: *house*/NN, plural: *houses*/NNS) are distinguished from proper nouns (singular: *Britain*/NP, plural: *Americas*/NPS)
- **Verbs** refer to actions. Base form: *go*/VB, past tense: *went*/VBD, past participle: *gone*/VBN, gerund: *going*/VBG, 3rd person singular present: *goes*/VBZ, other singular present: *am*/VBP. Special cases: modals *can*/MD, particles *switch on*/RP.
- **Adjectives** refer to properties of nouns. Regular: *green*/JJ, comparative: *greener*/JJR, superlative: *greenest*/JJS
- **Adverbs** refer to properties of verbs or adjectives. Regular: *happily*/*RB*, comparative: *ran faster*/RBR, superlative: *ran fastest*/RBS, wh-adverbs: *how*/WRB *fast*
- **Determiners** (also called **articles**) qualify or replace nouns. Regular: *the*/DT *house*, pre-determiner: *all*/PDT *the houses*, wh-determiner: *which*/WDT.
- **Pronouns** refer to previously mentioned nouns. Personal pronoun: *she*/PP, possessive pronoun: *her*/PP$, wh-pronoun: *who*/WP, possessive wh-pronoun: *whose*/WP$.
- **Prepositions** precede noun phrases or clauses and indicate their role in the sentence: *from*/IN *here*. Special case *to*/TO.
- **Coordinating conjunctions**: *and*/CC.
- **Numbers**: *17*/CD
- **Possessive marker**: *Joe 's*/POS
- **List item markers**: *A.*/LS
- **Symbols**: *$*/SYM
- **Foreign words**: *de*/FW *facto*/FW
- **Interjections**: *oh*/UH

Parts of Speech (POS) Tagging

- **POS tagging** can be treated as a **machine learning** problem
- Given a token and its features (e.g. the word itself, previous word, next word, prefix/suffix of the word), predict its POS tag
- A **trained model** can be found in NLTK

```python
import nltk
sentence = "Antoni Gaudí was a Spanish architect from Catalonia."
tokens = nltk.word_tokenize(sentence)
pos_tagged = nltk.pos_tag(tokens)
print(pos_tagged)
sentence = "Every morning I wake up at about seven o'clock."
tokens = nltk.word_tokenize(sentence)
pos_tagged = nltk.pos_tag(tokens)
print(pos_tagged)
```

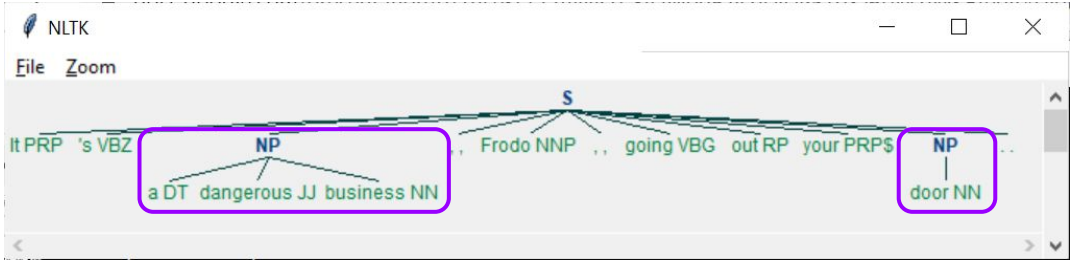
More about POS Tags: https://www.learntek.org/blog/categorizing-pos-tagging-nltk-python/

```
[('Antoni', 'NNP'), ('Gaudí', 'NNP'), ('was', 'VBD'), ('a', 'DT'), ('Spanish', 'JJ'),
('architect', 'NN'), ('from', 'IN'), ('Catalonia', 'NNP'), ('.', '.')]
[('Every', 'DT'), ('morning', 'NN'), ('I', 'PRP'), ('wake', 'VBP'), ('up', 'RP'), ('at',
'IN'), ('about', 'RB'), ('seven', 'CD'), ("o'clock", 'NNS'), ('.', '.')]
```

Chunk Grammar

- Chuck grammar is a combination of rules on how sentences should be chunked. It often uses **regular expressions**, or **regexes**.

```python
import nltk
lotr_quote = "It's a dangerous business, Frodo, going out your door."
words_in_lotr_quote = nltk.tokenize.word_tokenize(lotr_quote)
lotr_pos_tags = nltk.pos_tag(words_in_lotr_quote)
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(lotr_pos_tags)
tree.draw()
```



Two noun phrases (NP):

1. 'a dangerous business' has a determiner, an adjective, and a noun.
2. 'door' has just a noun.

Document Representation

- How do we **represent** a document in a program?
- We need to represent documents as a **feature vector** before we can **classify** them
- A commonly used representation is called the **bag-of-words model (bow)**
- Ignore any grammar, dependencies, punctuations, part-of-speech, etc.
- Order of word is assumed to be **not** important

Bag-of-words Model

- Once each document is represented by a **bag of words**, we can use a **vector** to represent each of them
- Firstly, let N be the total number of unique words (i.e. the **size** of our vocabulary*)
- We can use a vector of length N (N-dimension) to represent a document
- Each element in the vector corresponds to **one unique word**
- The element is 1 if the document contains the word, otherwise it is 0
- This is called **one-hot encoding**
- For example, if we have a vocabulary of (0="boy", 1="girl", 2="school"), a document with two words `boy` and `school` will be represented by the following vector:

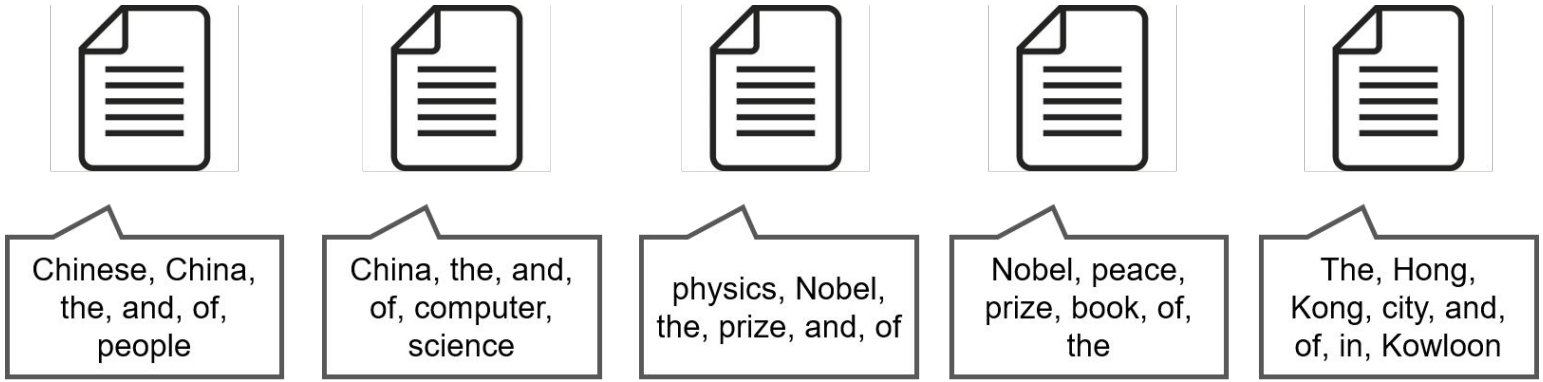
$$v=(1,0,1)$$

Vector Space Representation

- This **vector space representation** of document can be extended
- The value in the vectors can be assigned **different** meanings
  - Existence of individual words (0 or 1)
  - Number of times a word appears (0, 1, 2, ...)
  - A certain weighting assigned to the word (any real number)
- Intuitively, a word that appears more than the others should be more important
- However, some words are commonly used in general
  - articles (a, an, the)
  - pronouns (his, her, we, you, ...)
  - prepositions (of, in, over, ...)
- How can we assign a **meaningful weight?**

Term Weighting

- A commonly used **term weighting** scheme is called **tf-idf** (term frequency-inverse document frequency)
- Determine the **importance** of a word to a document by how often they appear across the whole corpus
- Consider a corpus with five documents:

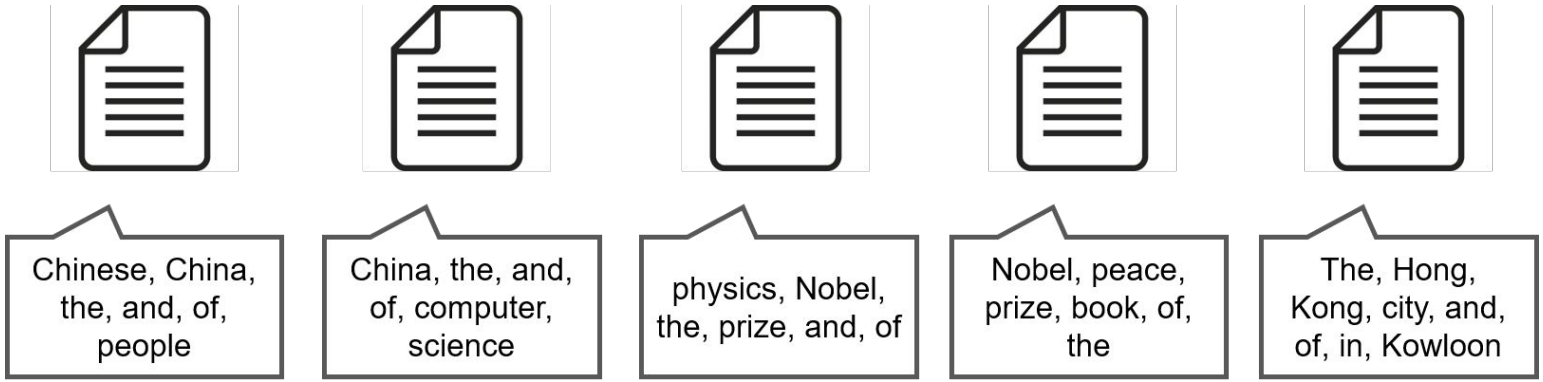


TF-IDF

- **Term Frequency (tf)**
  - tf(w,d) = Number of times w appears in d
- **Document Frequency (df)**
  - df(w) = Number of documents that contain w
- **Inverse Document Frequency (idf)**
  - Rare words has high idf, frequent words has low idf
  - idf(w) = log(N/df(w)), N is the number of documents
  - tf-idf(w,d) = tf(w,d) × idf(w)
- Remarks: There are different ways to define TF-IDF

TF-IDF

- Example: comparing importance of the word `china` vs. `and`
  - tf-idf(china,d_2) = 1×log(5/2) = 0.3979
  - tf-idf(and,d_2) = 1×log(5/4) = 0.0969



Transforming Documents in Scikit-learn `sklearn`

- In Python, you can use the CountVectorizer or TfidfVectorizer to convert texts into vectors

```python
from sklearn.feature_extraction.text import CountVectorizer
docs = [
    "CUHK is located in Shatin",
    "CUHK has a large campus",
    "Shatin is a district in the New Territories"
]
vectorizer = CountVectorizer()
vectorizer.fit(docs)  # Create the vocabulary
print(vectorizer.vocabulary_)

m = vectorizer.transform(["CUHK is in Shatin"]).todense()
print(m)
```

```
CountVectorizer()
{'cuhk': 1, 'is': 5, 'located': 7, 'in': 4, 'shatin': 9, 'has': 3, 'large': 6, 'campus': 0,
'district': 2, 'the': 11, 'new': 8, 'territories': 10}
matrix([[0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0]], dtype=int64)
```

Transforming Documents in Scikit-learn `sklearn`

- In Python, you can use the CountVectorizer or TfidfVectorizer to convert texts into vectors

```python
from sklearn.feature_extraction.text import TfidfVectorizer
docs = [
    "CUHK is located in Shatin",
    "CUHK has a large campus",
    "Shatin is a district in the New Territories"
]
vectorizer = TfidfVectorizer()
vectorizer.fit(docs)  # Create the vocabulary
print(vectorizer.vocabulary_)

m = vectorizer.transform(["CUHK is in Shatin"]).todense()
print(m)
```

```
TfidfVectorizer()
{'cuhk': 1, 'is': 5, 'located': 7, 'in': 4, 'shatin': 9, 'has': 3, 'large': 6, 'campus': 0,
'district': 2, 'the': 11, 'new': 8, 'territories': 10}
[[0.  0.5 0.  0.  0.5 0.5 0.  0.  0.  0.5 0.  0. ]]
```

Example: Using TF-IDF with Stems

1. For each sentence:
   - Apply tokenizer on the sentence
   - Apply stemming on the tokens
   - Update each word of the sentence with the stems
2. Apply the TfidfVectorizer on the transformed sentence

```
from nltk import word_tokenize
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
stemmer = PorterStemmer()
docs = [
  "An apple a day keeps the doctor away",
  "Eating an apple daily can improve digestion",
  "I eat an apple daily"
]
new_docs = []
```

Example: Using TF-IDF with Stems

```python
for sentence in docs:
  result = word_tokenize(sentence)
  new_sentence = ""
  for word in result:
    new_sentence += stemmer.stem(word) + " "

  print(sentence, "\n", new_sentence)
  new_docs.append(new_sentence)

vectorizer = TfidfVectorizer()
vectorizer.fit(new_docs)  # Create the vocabulary
print(vectorizer.vocabulary_)
m = vectorizer.transform(new_docs).todense()
print(m)
```

```
An apple a day keeps the doctor away
an appl a day keep the doctor away
Eating an apple daily can improve
digestion
eat an appl daili can improv digest
I eat an apple daily
i eat an appl daili
{'an': 0, 'appl': 1, 'day': 5,
'keep': 10, 'the': 11, 'doctor': 7,
'away': 2, 'eat': 8, 'daili': 4,
'can': 3, 'improv': 9, 'digest': 6}
```

```
[[0.24743277 0.24743277 0.4189401  0.         0.         0.4189401
  0.         0.4189401  0.         0.         0.4189401  0.4189401 ]
 [0.26806191 0.26806191 0.         0.45386827 0.34517852 0.
  0.45386827 0.         0.34517852 0.45386827 0.         0.        ]
 [0.43370786 0.43370786 0.         0.         0.55847784 0.
  0.         0.         0.55847784 0.         0.         0.        ]]
```

Pros and Cons of using NLP Preprocessing Tasks

Pros

- Static representation
- Simple and interpretable models
- Baseline for traditional ML tasks
- A better representation compared to those descriptive information (date, time, source, topic, … etc)

Cons

- Ignores semantic meaning and context
- Sparsity and high dimensionality
- Fail to handle synonyms and polysemy
- Sensitive to stop words and rare terms
- No out-of-vocabulary handling

Word Embeddings

word2vec

GloVe

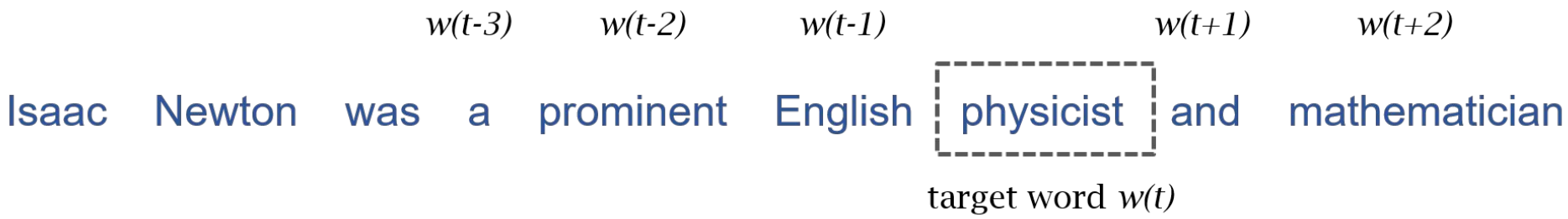
fastText

Word Embeddings

- Word vectors or embeddings are **dense vector** representation of words
- A type of **distributed representation** of words
- Word embeddings can be obtained by training a neural network on a large corpus of text data
- Training samples are generated from the corpus usually using a **sliding window** to define contexts
- Commonly used algorithms:
  - Word2Vec https://code.google.com/archive/p/word2vec/
  - GloVe https://nlp.stanford.edu/projects/glove/
  - fastText https://fasttext.cc/
  - AutoEncoder: https://yaronvazana.com/2019/09/28/training-an-autoencoder-to-generate-text-embeddings/

Introduction to Word2Vec

- **Basic idea**: A word's meaning is given by the words that frequently appear around it
- For a word w in a document, its context is the set of words that appear near it
- **"Near"** can be defined by **a sliding window of certain size**
- Word2vec (Mikolov et al. 2013) is an algorithm for learning word vectors



Word2Vec

- Word2vec obtains word embeddings by creating a new way of **supervised training** out of **unlabelled** data
- **Basic concept**:
  - Assume that we have a vocabulary of size N
  - For each word in this vocabulary, we initialize a random vector of dimension D (Usually D is about 100 to 300)
  - For each context window, we compute the **probability of each context word** given the **target word** using the **similarity** between their vectors
  - We keep adjusting the values of the vectors to maximize these probabilities

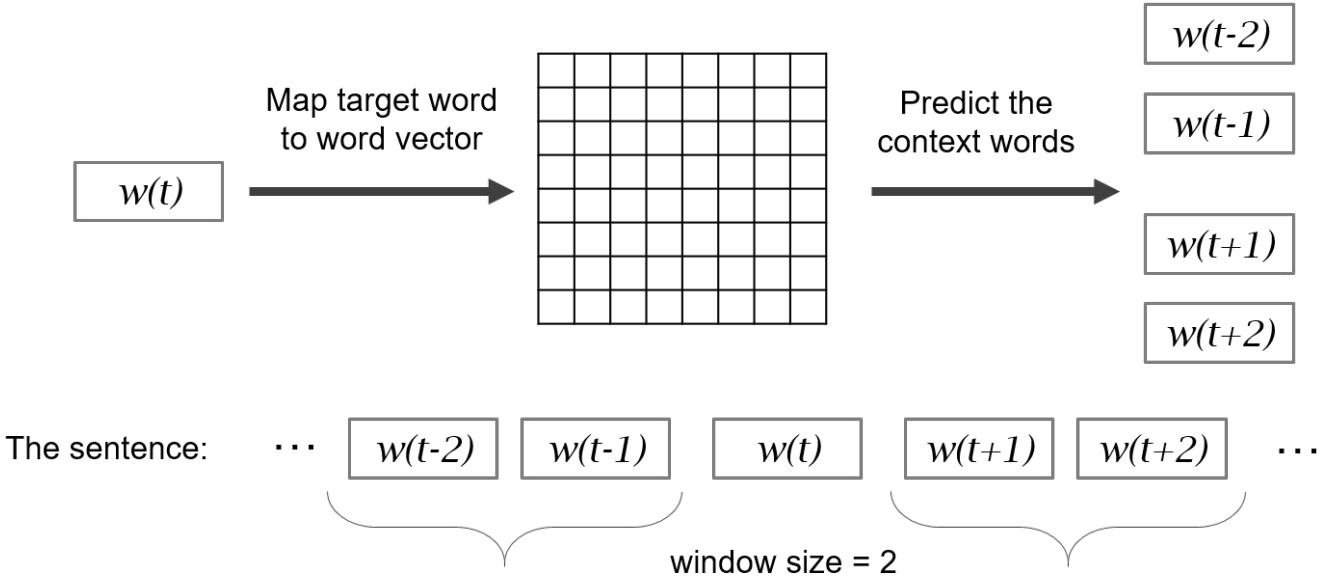
Word2Vec

- It turns out that **training** word vectors can be done using a neural network
- To train a neural network in a supervised way, we need to have some labelled data
- There are two ways to train a word vectors:

1. **Skip-gram (SG)**
   - For each context window, the **target word** (or **center word**) is the **input**
   - The context words on the left and right are the **output**
2. **Continuous Bag of Words (CBoW)**
   - For each context window, the **context words** are the **input**
   - The **target word** is the **output**

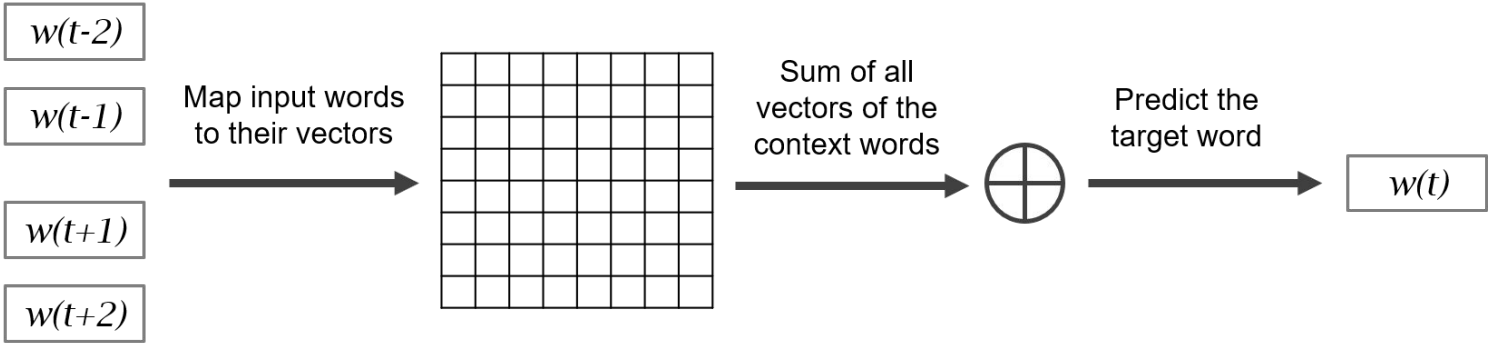
Skip-gram Model

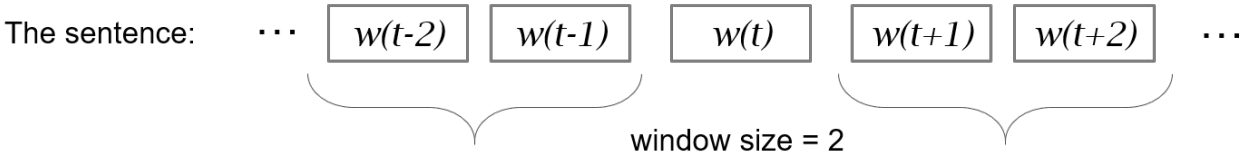
- In the Skip-gram model, the neural network is trained by using the **target word as input**, and the **context words as output**
- Task: train the model to output most likely context words given a target word



Continuous Bag-of-Words Model

- In the CBoW mode, the neural network is trained by using the **context words as input**, and the **target word as output**
- Task: using the context words to **predict** which would be the missing target word





Word Embeddings

- The trained word vectors show some interesting characteristics

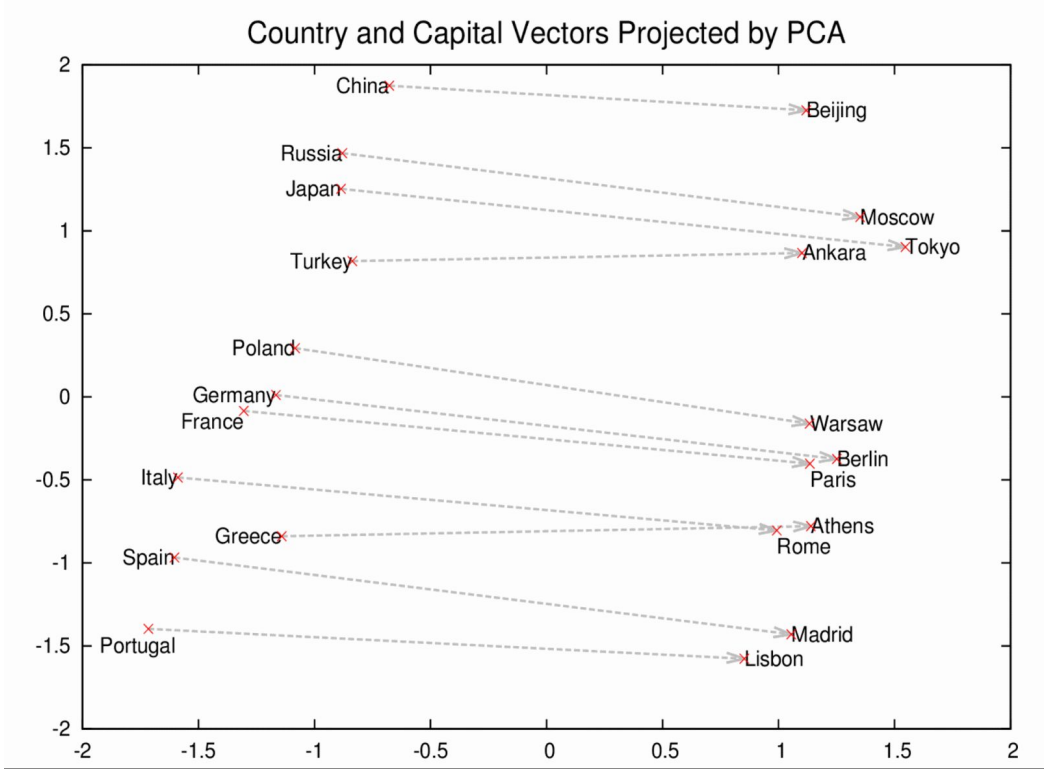


Image credit: [Distributed Representations of Words and Phrases and their Compositionality](#) (Mikolov et. al 2013)

Word2Vec in Python

- You can play with word vectors using the gensim Python package
- Obtain pre-trained word vectors from https://drive.google.com/file/d/0B7XkCwpI5KDYNlNUTTlSS21pQmM/edit?usp=sharing (Note: the file is 1.5GB compressed)

```python
# installation: conda install -c anaconda gensim --yes
from gensim.models.keyedvectors import load_word2vec_format
model = load_word2vec_format('GoogleNews-vectors-negative300.bin', binary=True)
# Get vector similarity
model.similarity("big", "huge")
# Get the vector of the word "great" (a 300-d vector)
model["great"]
# Find another word that is most similar to the given words
model.most_similar(["apple", "orange"])
```

```
0.78098553
array([ 7.17773438e-02,  2.08007812e-01, -2.84423828e-02,  1.78710938e-01,
        1.32812500e-01, -9.96093750e-02,  9.61914062e-02, -1.16699219e-01,
       -8.54492188e-03,  1.48437500e-01, -3.34472656e-02, -1.85546875e-01,
        4.10156250e-02, -8.98437500e-02,  2.17285156e-02,  6.93359375e-02,
        1.80664062e-01,  2.22656250e-01, -1.00585938e-01, -6.93359375e-02,
       ...
       -2.89062500e-01, -4.80957031e-02, -1.99218750e-01, -7.12890625e-02,
        6.44531250e-02, -1.67968750e-01, -2.08740234e-02, -1.42578125e-01],
      dtype=float32)
[('participant_LOGIN', 0.6436611413955688), ('peach', 0.6176683902740479), ('pear',
0.6049532890319824), ('olive', 0.6037271618843079), ('orangy', 0.5989357233047485),
('apples', 0.597709059715271), ('berry', 0.5965641140937805), ('apricot',
0.5942509174346924), ('strawberry', 0.5884400010108948), ('calico_lobsters_respectively',
0.5869752168655396)]
```

Introduction to Image Data

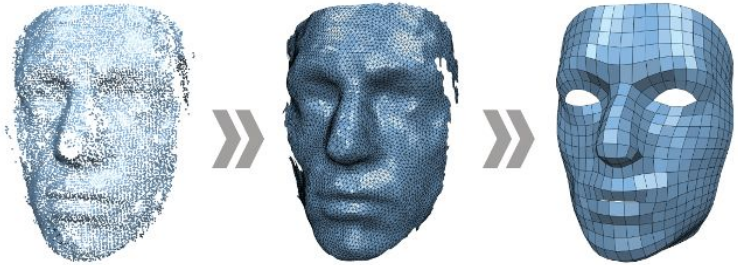
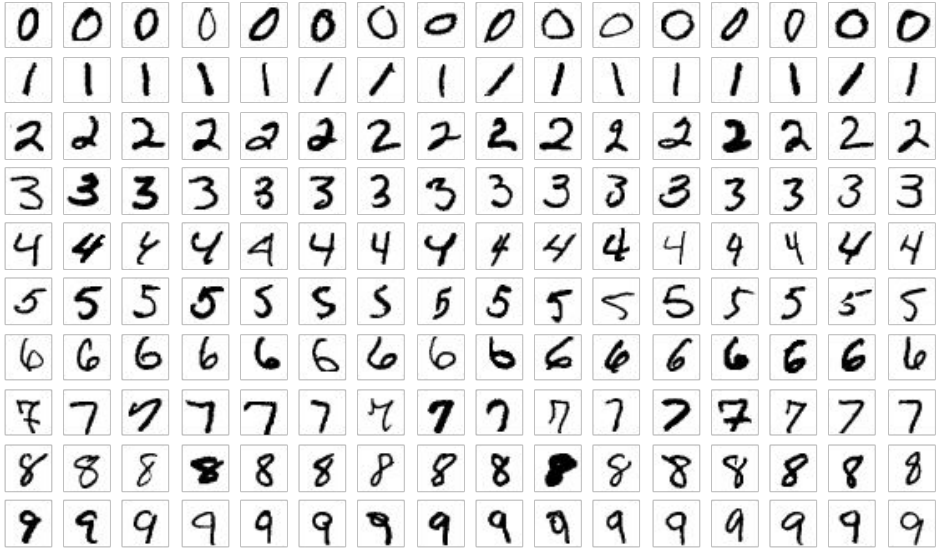


Image credit: Skyfi Labs

Image Classification Examples

- Some well-known image classification tasks:
  - MNIST Hand-written digits classification
  - ImageNet Large Scale Visual Recognition Challenge

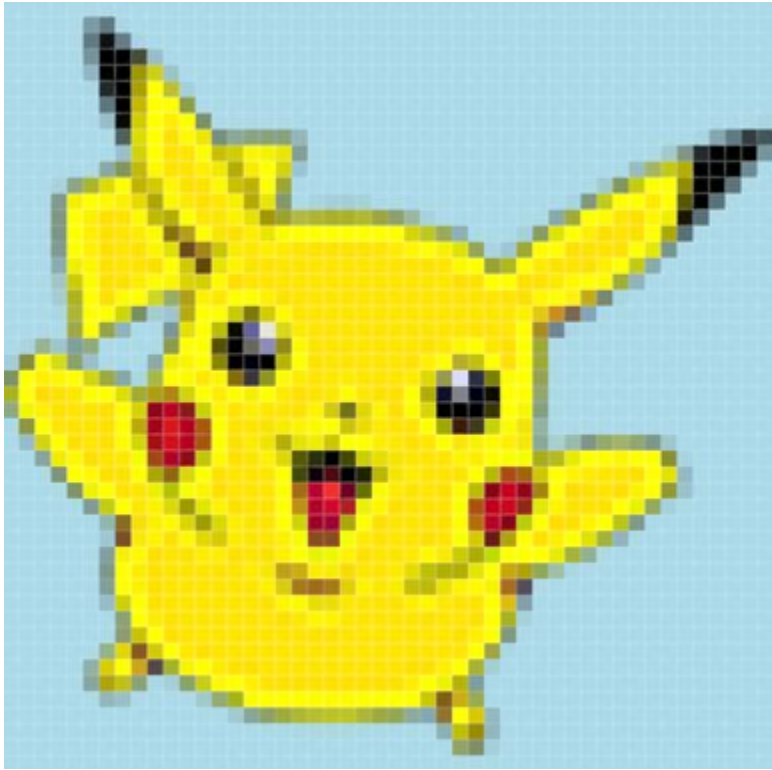


Challenges in Image Classification

1. **High dimensionality**
   - Images are represented digitally as a matrix of **pixels**
   - If we use **RGB** to represent the colour of a pixel, a 300 x 300 image has 270,000 values
2. **Representation**
   - Representing an image using a feature vector of all the pixels is of little use
   - The scale and perspective of the object inside an image can vary a lot
3. **Noise**
   - A picture of an object usually have something in the background
   - An object may appear with another irrelevant object in the same image

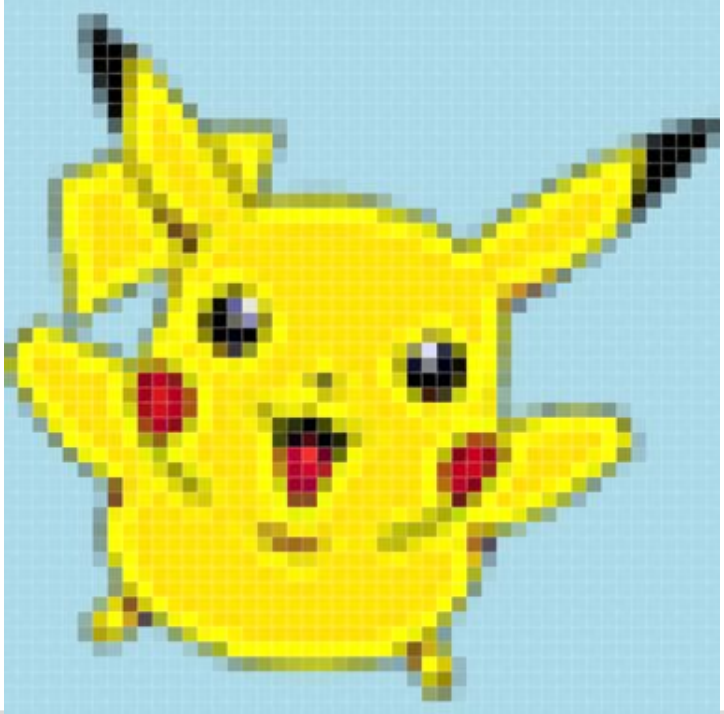
What is an Image?

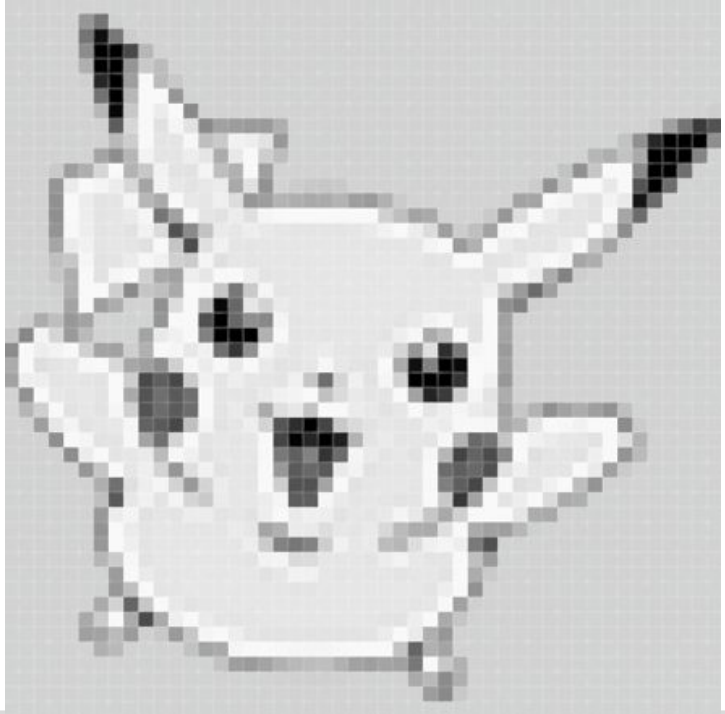
- An image is represented digitally by **pixels** (picture elements)
- Each pixel contains the values of the colour of the corresponding component of the image
- Resolution refers to the **number of pixels** used to represent the image
- The values of the pixels can represent **the intensity of light**, **the RGB values**, **hue**, **saturation**, **brightness**, etc.



What is an Image?

- A **grayscale** version of the previous image
- Each pixel has a value between 0 and 255





RGB (Red, Green, Blue) Channel

- One of the most commonly used colour model
- Each pixel is represented by **three integer values**, each of **8 bits (0 to 255)**
- This allows 16,777,216 (256^3) combinations for a pixel

(R, G, B) = (84, 130, 53)
HEX = 548235

Why is image classification difficult?

- Semantics is lost in an image represented as a matrix of pixels
- We need to bridge the gap between pixels and meanings
- What are the features that can be used to represent an image?

What a human sees

1	9	2	45	76	9	5	2
0	99	32	4	1	21	2	97
0	7	43	35	64	43	35	41
0	36	4	72	8	45	12	36
86	12	57	7	6	6	66	8
41	1	2	26	5	12	20	0
3	23	4	96	4	2	93	46
21	23	8	3	3	0	12	2

What a computer sees

Representation and Feature Extraction

- In order to perform computation on images, we need an **appropriate representation** of the images.
- Some representations are:
  - Use the values of the **raw pixels**
  - Use **keywords (metadata)** to describe an image, and thus an image can be represented as a bag of words
  - Extract **visual features** from the image and use a vector of features to represent an image
- Finding a **good representation** is hard

Image Features

- A **feature** is a piece of information relevant for solving a certain computational task
- In **computer vision**, features can be pixels, points, edges, shapes, or other more complex **visual structures**
- **Feature detection** – the process of identifying features in an image that is representative of the image itself
- Reference: https://en.wikipedia.org/wiki/Feature_detection_(computer_vision)

Feature Extraction from Image

Edge Detection

https://en.wikipedia.org/wiki/Edge_detection

Object Detection

Sobel Edge Detection Methods

- Use convolution mask and based on discrete approximations to differential operators
- Differential operations measure the rate of change in the image brightness function
- Some operators return orientation information. Other only return information about the existence of an edge at each point

$$y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \qquad x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Edge magnitude: $\sqrt{x^2 + y^2}$ Edge direction: $\tan^{-1}\left[\frac{y}{x}\right]$

Example: Edge Detection with Sobel and Canny

```python
# installation under Anaconda Powershell Prompt:
# conda install -c conda-forge opencv
import cv2
import numpy as np
# load the image
img = cv2.imread('tiger.jpg')
# convert to graycsale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# blur the image for better edge detection
blur = cv2.GaussianBlur(gray, (3,3), 0)
# combined X and Y Sobel Edge Detection
sobel = cv2.Sobel(src=blur,ddepth=cv2.CV_64F,dx=1,dy=1,ksize=5)
# output the Sobel edge detection result
cv2.imwrite('Sobel_tiger.png', sobel)
# Canny Edge Detection
edges = cv2.Canny(image=blur, threshold1=100, threshold2=200)
# output the Canny edge detection result
cv2.imwrite('Canny_tiger.png', edges)
```

Reference: https://learnopencv.com/edge-detection-using-opencv/

Image Descriptors

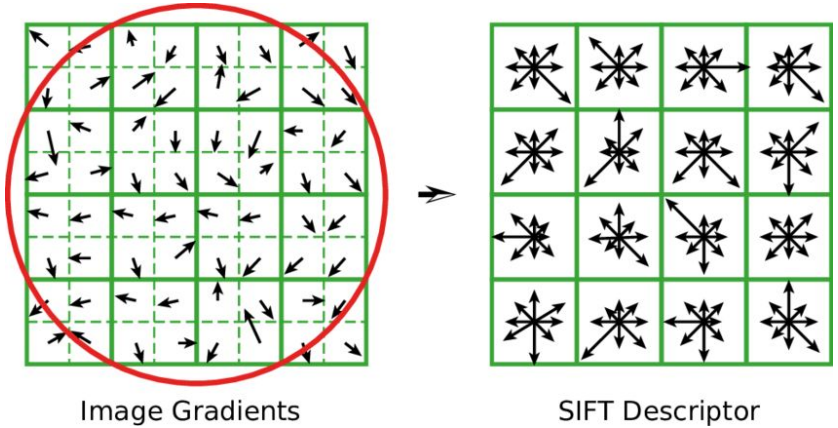


Image credit: ResearchGate

Image Descriptors

- Some image descriptors extract basic visual features, including
  - Color descriptors (e.g. Histogram)
  - Texture descriptors (e.g. local binary pattern)
  - Edge or shape descriptors (e.g. HOG, Canny edges)
- Some image descriptors use keypoints to describe its surrounding regions
  - E.g. SIFT, SURF descriptors
- Some image descriptors also use deep learning to create image embeddings
  - We will discuss these in the next set of slides

Scale-Invariant Feature Transform (SIFT)

- Published by David Lowe (now at Google) in 1999/2004
- An algorithm for detecting and describing **local features** in images
- One of the most widely used feature extraction algorithm in computer vision
- Able to detect features when the **scales** and **rotations** are different
- Reference: Distinctive Image Features from Scale-Invariant Keypoints

SIFT Problem Definition

- What are the criteria of **good features**?
  a. Distinctive
  b. Easy to extract
  c. Invariant to noise, illumination ,scaling, rotation, viewing direction
  d. Easy to match against a large database of features

SIFT

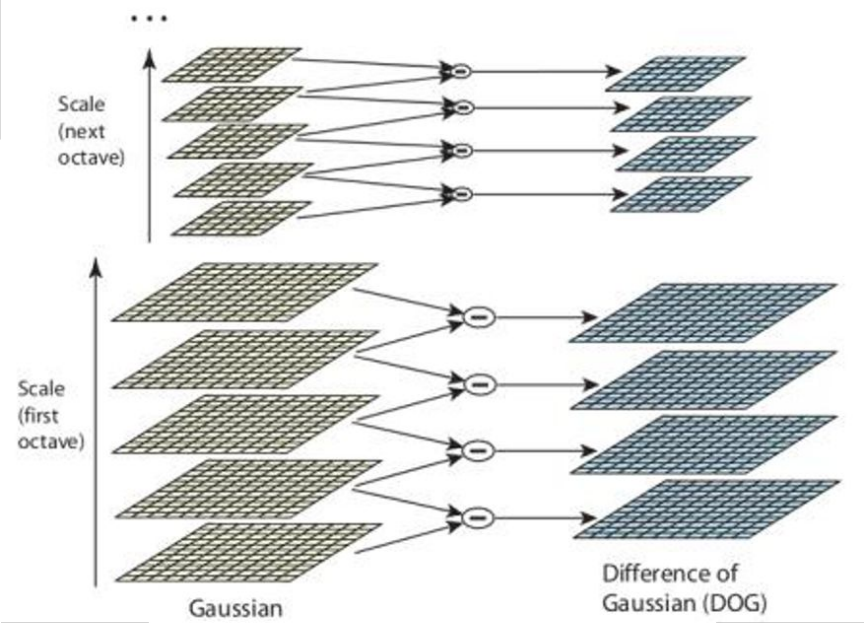
- SIFT is an approach for detecting (determine what is a feature) and describing local features with the following steps:

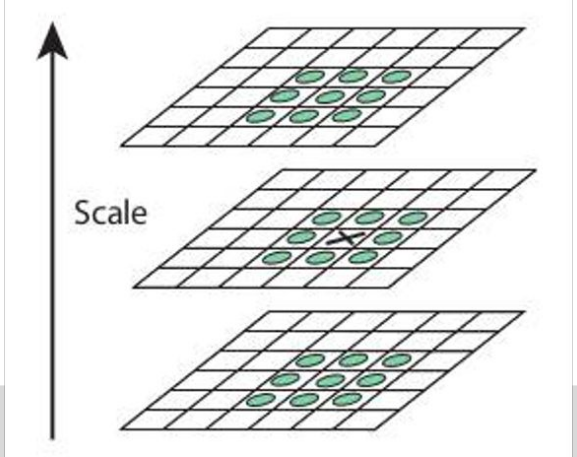
1. Scale-space extrema detection
2. Keypoint localization
3. Orientation assignment
4. Generation of keypoint descriptors

- Reference: http://docs.opencv.org/trunk/da/df5/tutorial_py_sift_intro.html

SIFT

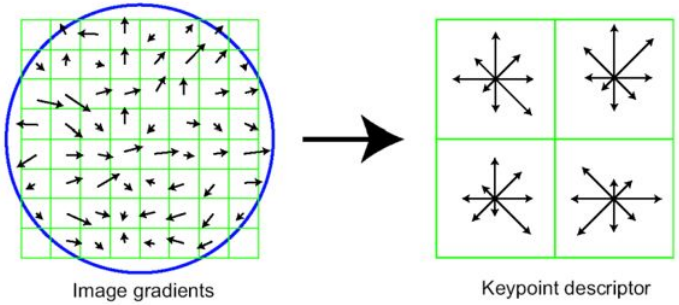
1. **Scale-space extrema detection**
   - Search for stable features across different scales
   - An image is rescaled and smoothed by a Gaussian (blurring) filter
   - Find out interesting points by selecting the local maxima or minima among different scales
2. **Keypoint localization**
   - Remove points with low contrasts or edges (not necessary interesting features)





SIFT

3. **Orientation assignment**
   - Assign an orientation to each keypoint, so that the features detected are invariant to image rotation
4. **Generation of keypoint descriptors**
   - A **16x16** neighbourhood around the keypoint is taken, and a histogram of orientations of the gradients are calculated
   - Finally, each keypoint is represented by **a vector of length 128**



SIFT

- Interesting points can be used to **match objects** across different images

Example: SIFT in OpenCV

```python
# installation under Anaconda Powershell Prompt:
# conda install -c conda-forge opencv
import cv2
import numpy as np
# load the image
img = cv2.imread('star-ferry.png')
# apply color space conversion
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
# create and learn the SIFT descriptor
sift = cv2.SIFT.create()
# kp: keypoints, des: descriptor
kp, des = sift.detectAndCompute(gray,None)
flag = cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
img = cv2.drawKeypoints(gray, kp, img, flags=flag)
# output the image with keypoints
cv2.imwrite('keypoints.png', img)
```

```
True
```

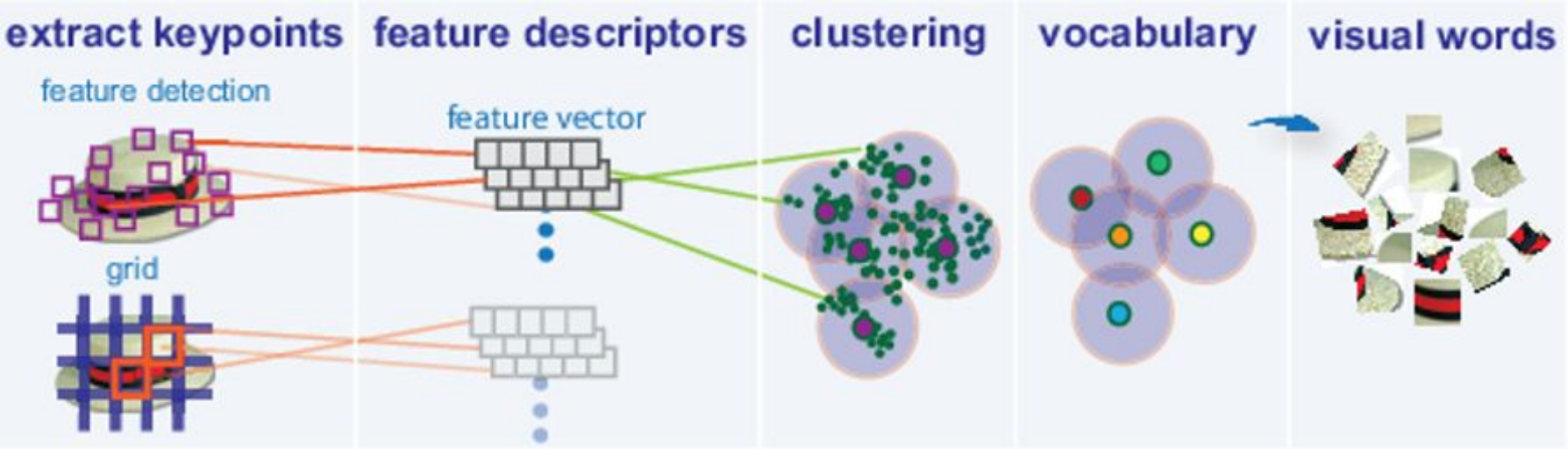
More about KeyPoint Descriptors

Field	Type	Description	Example
`angle`	`float`	keypoint orientation	`> kp[0].angle` `339.1086120605469`
`class_id`	`int`	object id (if the keypoints need to be clustered by an object they belong to)	`> kp[0].class_id` `-1`
`octave`	`int`	pyramid octave in which the keypoint has been detected	`> kp[0].octave` `10159103`
`pt`	`Point2f`	x & y coordinates of the keypoint	`> kp[0].pt` `(5.81308794021,111.902534484)`
`response`	`float`	keypoint detector response on the keypoint (that is, strength of the keypoint)	`> kp[0].response` `0.015389029867947102`
`size`	`float`	keypoint diameter	`> kp[0].size` `3.281869649887085`

Reference: https://docs.opencv.org/4.x/d2/d29/classcv_1_1KeyPoint.html

Visual Bag-of-Words

- In text classification, a document can be represented by a **bag of words**
- Similar approach can be used in image classification
- Reference:
  http://www.mathworks.com/help/vision/ug/image-classification-with-bag-of-visual-words.html

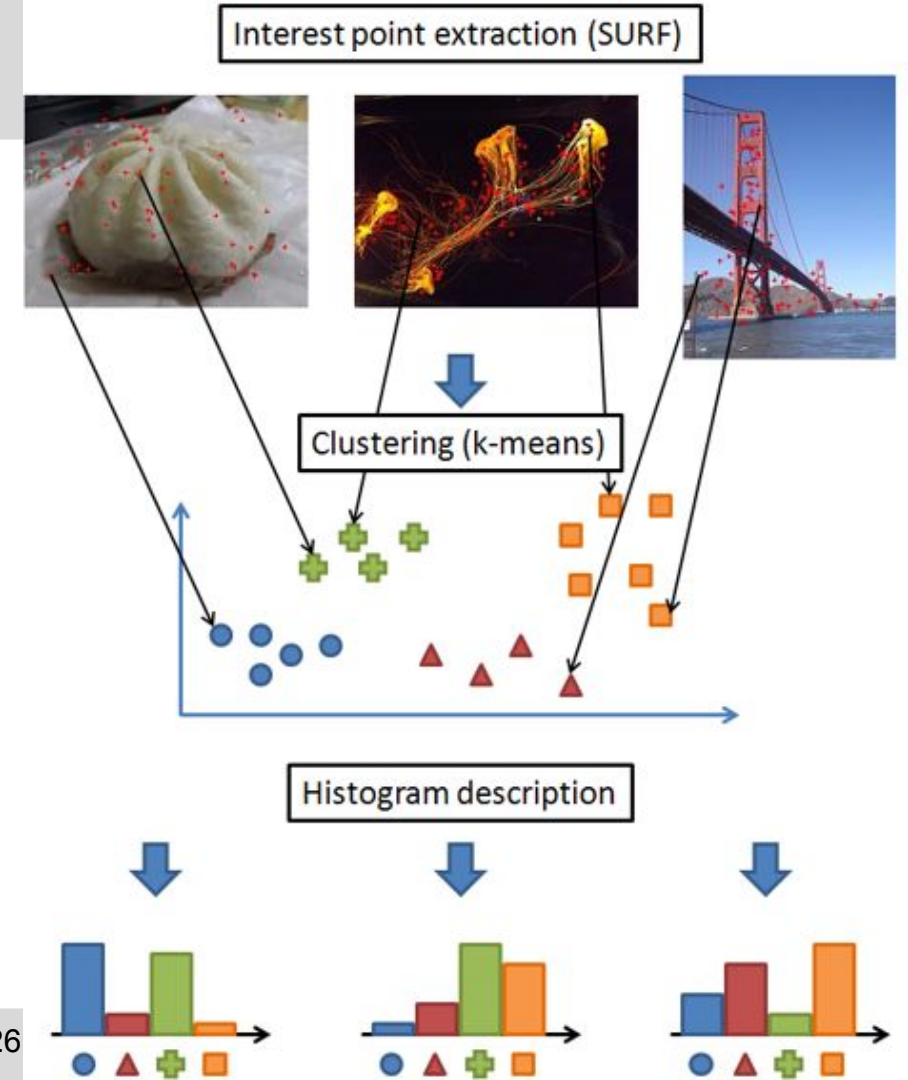


Steps for Visual Bag-of-Words

1. Extract **keypoints** from images using SIFT
2. **Cluster** the keypoints to create a **visual dictionary** using a clustering algorithm (e.g. K-means)
3. For a given image, check the cluster to which each keypoint is in, and create a **histogram** of the distribution of clusters
4. Images are then represented by **vectors by normalizing the histograms**

Visual Bag-of-Words

- Reference: http://www.olivier-augereau.com/blog/?p=358



Limitations for SIFT

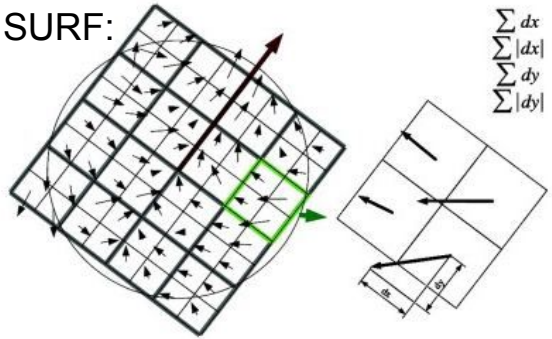
1. SIFT aims at identifying interesting points automatically, but keypoints are still relatively **primitive** and are not usually good enough for **classification tasks**
2. The visual BoW approach does not consider **spatial relations** among visual words
3. SIFT cannot be easily adapted to **different domains** to extract distinct features, as it is not based on **machine learning**
4. SIFT is computationally expensive

- We need better algorithms to extract image features

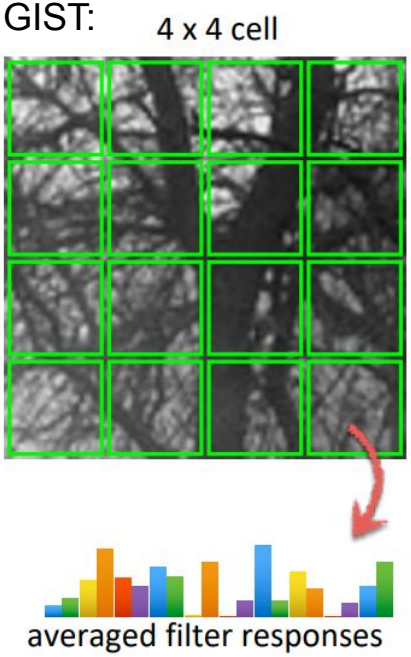
Other Famous Descriptors

1. Speeded Up Robust Features (SURF) Descriptor
2. Multi-Scale Oriented Patches (MOPS) Descriptor
3. GIST descriptor
4. Histogram of Oriented Gradients (HOG) Descriptor

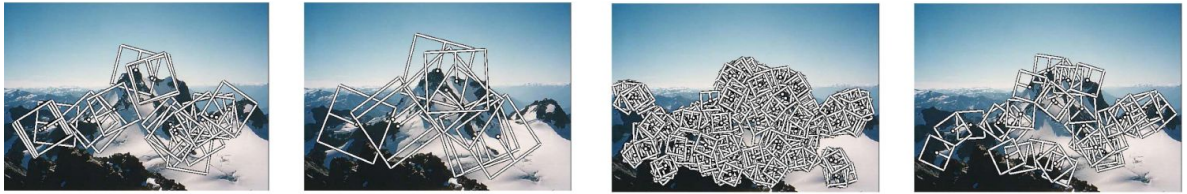
SURF:



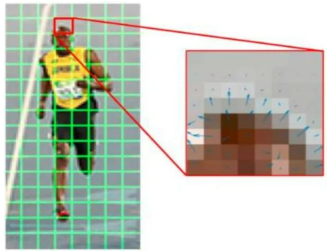
GIST:



MOPS:



HOG:



Note: MOPS and GIST are not implemented in the OpenCV, Scikit packages.

Example: HOG Descriptor in OpenCV

```python
from skimage.io import imread
from skimage.transform import resize
from skimage.feature import hog
import matplotlib.pyplot as plt
import cv2
# load the image
img = imread('star-ferry.png')
# apply color space conversion
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
# resizing image
resized_img = resize(gray, (128*4, 64*4))
# creating hog features
fd, hog_image = hog(resized_img, orientations=9,
pixels_per_cell=(8, 8), cells_per_block=(2, 2),
visualize=True)
# save the images
plt.imsave("resized_img.jpg", resized_img)
plt.imsave("hog_image.jpg", hog_image, cmap="gray")
```

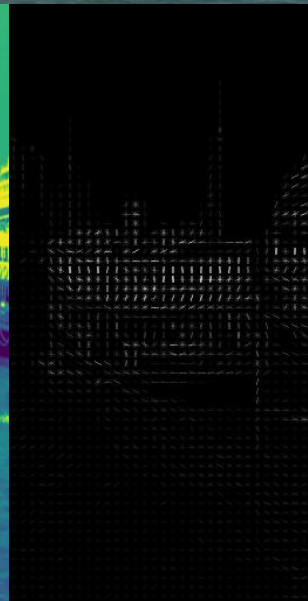


Image Tensors

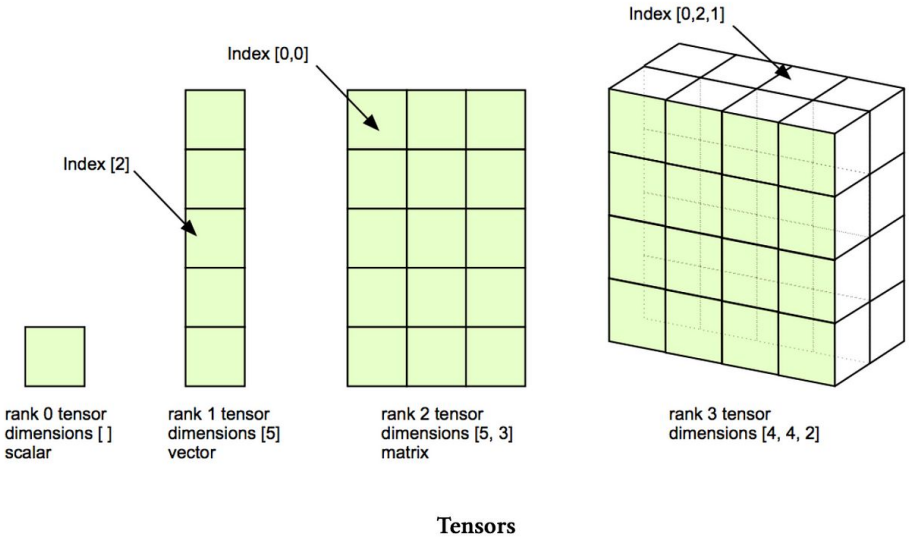


Image credit: Google Image

Image Tensors

- Image tensors are **multi-dimensional numerical arrays** (usually 3D) that represent images in a structured format suitable for processing in deep learning frameworks (e.g. PyTorch). They generalize matrices (2D arrays) to higher dimensions, encoding pixel data, color channels, and batch information.

Example: Image Tensors

```python
import torch
from PIL import Image
from torchvision import transforms

img = Image.open("cat.jpg")

# Convert to tensor
transform = transforms.Compose([
  transforms.Resize((224, 224)),    # Resize
  transforms.ToTensor(),            # PIL → Tensor (C, H, W)
  transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
                                    # ImageNet stats
])

tensor = transform(img)             # Shape: (3, 224, 224)
batch_tensor = tensor.unsqueeze(0)  # Add batch dim: (1, 3, 224, 224)
```

Common Tensor Operations

1. Reshaping

```
# PyTorch: (N,H,W,C) → (N,C,H,W)
tensor = tensor.permute(0, 3, 1, 2)
```

2. Cropping

```
# Crop a region
cropped = tensor[:, :, 50:150, 50:150]
```

3. Batch processing

```
# Forward pass on a batch
output = model(batch_tensor)
```

Common Challenges with Image Tensors

1. **Memory Usage**
   - GPU is preferred for deep learning tasks
   - If you only have a CPU, just try smaller batches with a sample of the original dataset.
2. **Channel Ordering**
   - PyTorch: (N, **C**, H, W) (channels-first)
   - TensorFlow: (N, H, W, **C**) (channels-last)
   - Better to stick with PyTorch, or transpose tensors when switching frameworks.
3. **Normalization Consistency**
   - Mismatched normalization (e.g., [0,1] vs. [-1,1]) hurts model performance.

Introduction to Audio Data

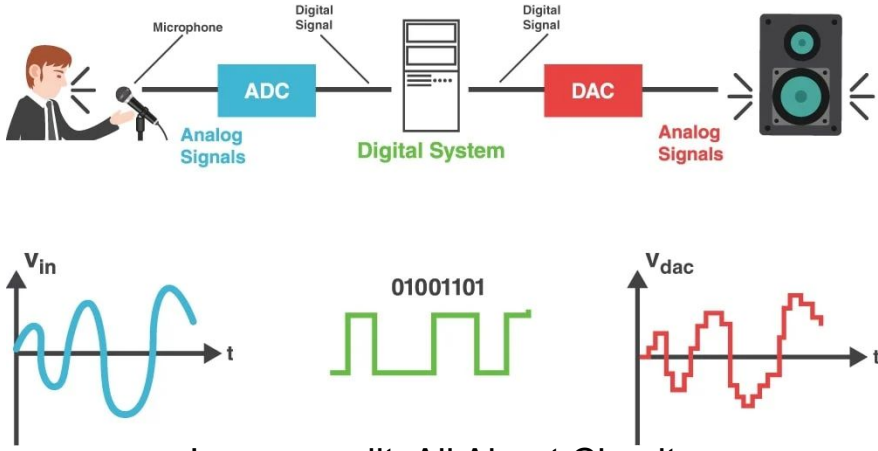


Image credit: All About Circuits

Audio Signal Processing

- Audio signal processing focuses on the computational methods for intentionally altering auditory signals or sounds, in order to achieve some particular goals (our goals, audios to numbers)

Image credit: https://www.slideshare.net/aas88/lecture-onaudio

Audio Signals

- **Digital format**: The pressure waveform is a sequence of symbols, unusually binary number e.g. use of microprocessor
- **Analog format**: The smooth wave of energy represented by a continuous stream of data e.g. electronic sound

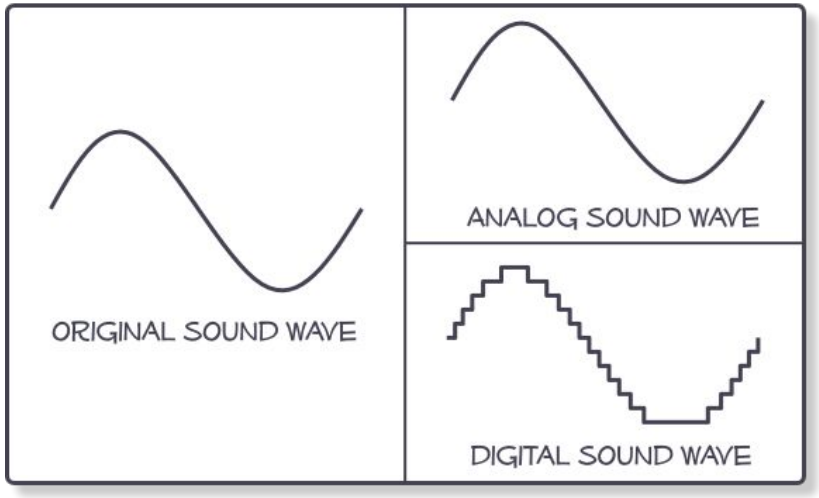


Image credit: http://www.centerpointaudio.com/Analog-VS-Digital.aspx

Basic Tasks for Audio Signal Processing

1. Read and write audio files in different formats (WAV, MP3, WMA, FLAC etc.).
2. Play the sound on your computer.
3. Represent the sound as a waveform, and process it: filter, resample, build spectrograms etc.

```
# installation under Anaconda Powershell Prompt:
# step 1: pip install soundfile
# step 2: conda install -c conda-forge pysoundfile --y
import soundfile as sf
# y is audiodata, a 2D (frames x channels) NumPy array
# sr is sample rate of the audio file
y, sr = sf.read('preamble.wav')
print(y.shape, y.dtype, sr)
sf.write('out.wav', y, sr)
```

- Other libraries: Librosa, SoundDevice, PyDub, TorchAudio, FFMpeg, GStreamer and LibSoX

Reference: https://www.it-jim.com/blog/audio-processing-basics-in-python/

Example: Basic Operations with FFmpeg

```python
# installation under Anaconda Powershell Prompt:
# step 1: conda install conda-forge::ffmpeg --y
# step 2: conda install conda-forge::ffmpeg-python --y
# ffmpeg is the implementation, while ffmpeg-python is the script-like wrapper
import ffmpeg
# extract 10-20s and convert from mp3 to wav
start_time = '00:00:10' # Start time for trimming (HH:MM:SS)
end_time = '00:00:20'   # End time for trimming (HH:MM:SS)
(
  ffmpeg.input("Greensleeves.mp3", ss=start_time, to=end_time)
  .output("Greensleeves_trim.wav")
  .run()
)
```

- You can also supply a video (.mp4) to ffmpeg and extract audio (mp3) / image (.jpg, .png) from the video

Reference: https://www.bannerbear.com/blog/how-to-use-ffmpeg-in-python-with-examples/

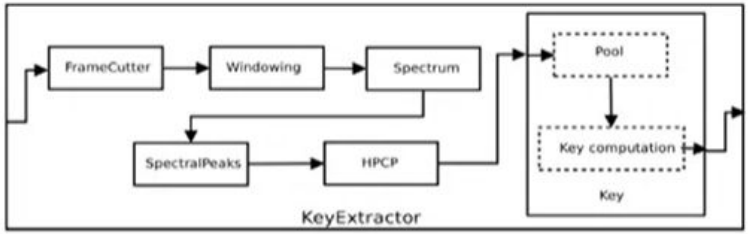
Basic Parameters of Sound Files

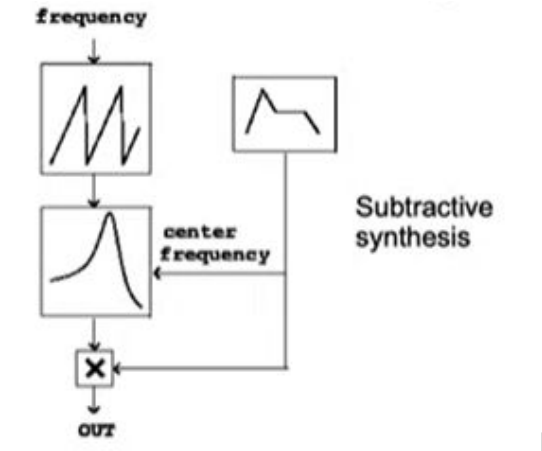
1. **Number of channels**: One channel: mono; Two channels: stereo
2. **Sample width**: Number of bytes for each sample
3. **Frame rate / sample rate / sample frequency**: the number of samples per second that are taken of a waveform to create a discrete digital signal
   - E.g. 44,100 Hz, the standard CD quality, 44,100 sample values in each second
4. **Number of frames**: the total number of frames
5. **Values of a frame**: a binary format storing the actual audio signal

- Duration of audio = Number of frames / frame rate

Some Common Goals in Audio Processing Signals

1. **Data compression**: The technique of reducing storage space
2. **Create effects and transformations**: e.g. use in music and film production
3. **Sound synthesis**: The technique of generating sound, using electronic hardware or software, from scratch
4. **Sound description**: The technique of analyzing the sound signal, to generate a model to describe the concept of some sound





Reference:
https://www.coursera.org/lecture/audio-signal-processing/introduction-to-audio-signal-processing-fHha1

Classic Audio Signal Processing

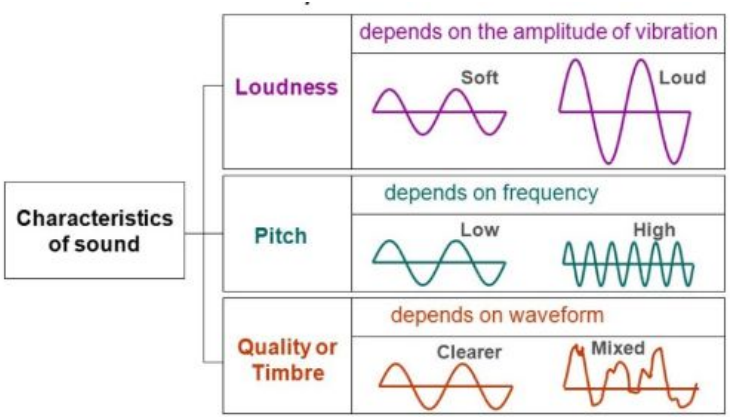


Image credit: ProProfs

Some Common Features from Sound Data

1. Pitch
2. Timbre
3. Sound power, sound intensity
4. Thresholds of pain, threshold of hearing
5. Loudness
6. Intensity level
7. Complex sound
8. Spectral centroid
9. Spectral rolloff
10. Spectral bandwidth
11. Zero-crossing rate
12. Mel-Frequency Cepstral Coefficients(MFCCs)
13. Chroma feature

- See: https://www.kaggle.com/code/ashkhagan/audio-signal-processing-librosa, https://www.kdnuggets.com/2020/02/audio-data-analysis-deep-learning-python-part-1.html

Pitch

- **Pitch** refers to how high or low a sound is perceived by the ear. It is determined by the frequency of a sound wave, measured in Hertz (Hz) (vibrations per second). For example:

1. A standard tuning note is A4 (440 Hz).
2. A deep male voice (男低音) might sing around 100 - 300 Hz, while a soprano (女高音) can reach 1000 Hz or higher.

Example: Average Pitch

```python
# Remark: Up to July 2025, Librosa only works up to python version 3.12
import librosa
import numpy as np

# Load MP3 file (librosa automatically converts to WAV internally)
audio_path = "YOUR_SONG.mp3"
y, sr = librosa.load(audio_path, sr=44100)  # y: audio samples, sr: sample rate

# Extract pitch using PYIN
f0, voiced_flag, voiced_probs = librosa.pyin(y, fmin=librosa.note_to_hz('C2'),
fmax=librosa.note_to_hz('C7'))

# Get mean pitch (ignoring unvoiced frames)
mean_pitch = np.nanmean(f0) if not np.all(np.isnan(f0)) else None
print(f"Mean Pitch: {mean_pitch} Hz")
```

```
Mean Pitch: 83.71545094378475 Hz
```

Running Python 3.12 in Anaconda Python 3.13

- **Virtual Environment** is an isolated workspace that allows you to manage dependencies (libraries/packages) separately for different projects. This prevents conflicts between package versions and keeps your system Python installation clean.
- Create a new conda environment with Python 3.12

```
conda create -n py312 python=3.12
```

- Activate the environment

```
(base) > conda activate py312
(py312) >
```

Running Python 3.12 in Anaconda Python 3.13

- Verify Python version

```
(py312) > python --version
Python 3.12.3
```

- Install relevant packages after activating the environment, for example,

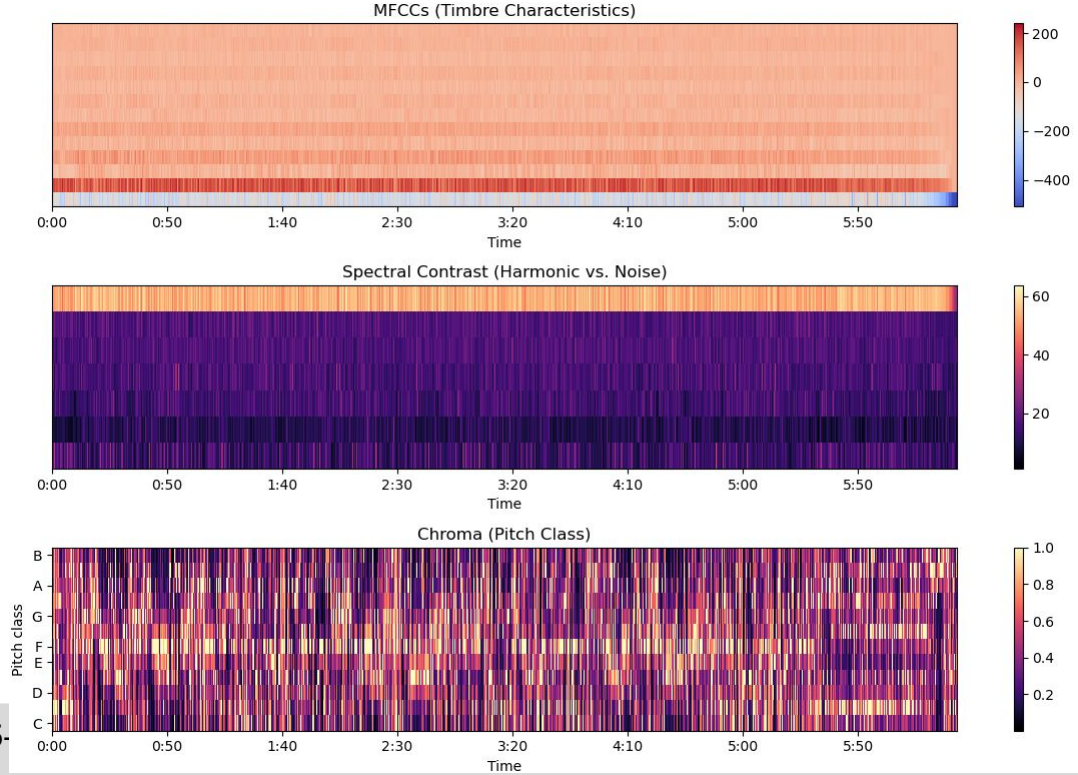
```
(base) > conda activate py312
(py312) > conda install conda-forge::librosa --y
```

- After using the environment, you wish to switch back to the base

```
(py312) > conda deactivate
(base) >
```

Timbre

- **Timbre** refers to the unique tone color or quality of a sound that distinguishes one instrument or voice from another, even when they play the same pitch at the same volume.
- Timbre makes music expressive and recognizable.



Example: Timbre

```python
# Remark: Up to July 2025, Librosa only works up to python version 3.12
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

def analyze_timbre(mp3_path):
    # Load MP3 file
    y, sr = librosa.load(mp3_path, sr=None)  # y = audio data, sr = sample rate

    # Extract timbre-related features
    mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)  # MFCCs, Harmonic vs. noise
    spectral_contrast = librosa.feature.spectral_contrast(y=y, sr=sr)
    chroma = librosa.feature.chroma_stft(y=y, sr=sr)  # Pitch class profile

    # Plot timbre features
    plt.figure(figsize=(12, 8))
    plt.subplot(3, 1, 1)
    librosa.display.specshow(mfccs, x_axis='time')
```

Example: Timbre

```
    plt.colorbar()
    plt.title('MFCCs (Timbre Characteristics)')
    plt.subplot(3, 1, 2)
    librosa.display.specshow(spectral_contrast, x_axis='time')
    plt.colorbar()
    plt.title('Spectral Contrast (Harmonic vs. Noise)')
    plt.subplot(3, 1, 3)
    librosa.display.specshow(chroma, x_axis='time', y_axis='chroma')
    plt.colorbar()
    plt.title('Chroma (Pitch Class)')
    plt.tight_layout()
    plt.show()

# Example usage
analyze_timbre("YOUR_SONG.mp3")
```

Example: Vocal Separation from Song

- For details, you may refer to the original article: [Vocal Separation Using Nearest Neighbours and Median Filtering]

```
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt
import librosa
import librosa.display
import soundfile as sf

y, sr = librosa.load('16-TheRapsWell_CleanAlbumVersion_192kb.mp3', duration=120)

# And compute the spectrogram magnitude and phase
S_full, phase = librosa.magphase(librosa.stft(y))

```

Example: Vocal Separation from Song

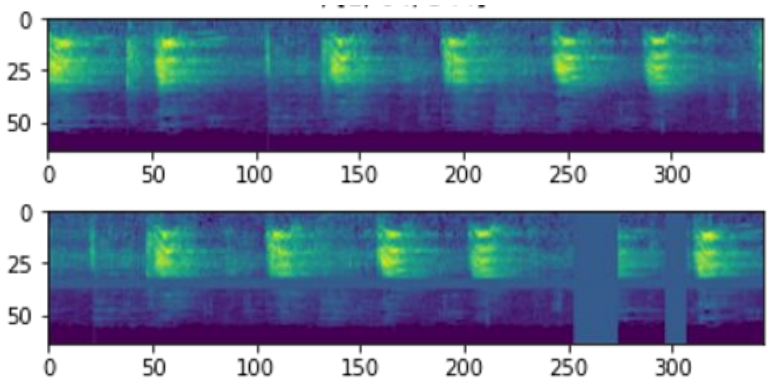
```
# We'll compare frames using cosine similarity, and aggregate similar frames by taking
# their (per-frequency) median value.
# To avoid being biased by local continuity, we constrain similar frames to be
# separated by at least 2 seconds.
# This suppresses sparse/non-repetitive deviations from the average spectrum, and
# works well to discard vocal elements.
S_filter = librosa.decompose.nn_filter(S_full, aggregate=np.median, metric='cosine',
width=int(librosa.time_to_frames(2, sr=sr)))

# The output of the filter shouldn't be greater than the input
# if we assume signals are additive.  Taking the pointwise minimum with the input
# spectrum forces this.
S_filter = np.minimum(S_full, S_filter)

# We can also use a margin to reduce bleed between the vocals and instrumentation
# masks.
# Note: the margins need not be equal for foreground and background separation
margin_i, margin_v = 2, 10
power = 2
mask_i = librosa.util.softmask(S_filter, margin_i * (S_full - S_filter), power=power)
mask_v = librosa.util.softmask(S_full - S_filter, margin_v * S_filter, power=power)
```

Example: Vocal Separation from Song

```
# Once we have the masks, simply multiply them with the input spectrum to separate the
# components
S_foreground = mask_v * S_full
S_background = mask_i * S_full

# Recover both the foreground and background audios
y_foreground = librosa.istft(S_foreground * phase)
y_background = librosa.istft(S_background * phase)

# Save to audio files
sf.write("foreground.wav", y_foreground, sr)
sf.write("background.wav", y_background, sr)

```

Reference: https://librosa.org/librosa_gallery/auto_examples/plot_vocal_separation.html

Mel Spectrogram and Masking



Steps for Audios to Numbers

1. Read audio from a file
2. Convert all audios to the same channel format
   - One channel: mono; Two channels: stereo
3. Standardize sampling rate
   - Sampling rate is the number of samples per second that are taken of a waveform to create a discrete digital signal
4. Resize to the same length
5. Data Augmentation: Time Shift
6. Mel Spectrogram, a model of sound information
7. Data Augmentation: Time and Frequency Masking

Reference:
https://towardsdatascience.com/audio-deep-learning-made-simple-sound-classification-step-by-step-cebc936bbe5

Read Audio Using TorchAudio

```python
# installation under Anaconda Powershell Prompt: conda install -c pytorch torchaudio
import torchaudio
def open(audio_file):
  sig, sr = torchaudio.load(audio_file)
  # sig is audiodata
  # sr is sample rate
  return (sig, sr)

```

Convert Mono Channel to-and-from Stereo Channels

- To keep all files having the same dimension

```python
def rechannel(aud, new_channel):
   sig, sr = aud
   if (sig.shape[0] == new_channel):
     # Nothing to do
     return aud
   if (new_channel == 1):
     # Convert from stereo to mono by selecting only the first channel
     resig = sig[:1, :]
   else:
     # Convert from mono to stereo by duplicating the first channel
     resig = torch.cat([sig, sig])
   return ((resig, sr))
```

Standardize Sampling Rate

- Keep all audio to the same sampling rate, everything need to be the same dimension, e.g. some 48000Hz (1 sec has 48000 values), some 44100Hz

```
def resample(aud, newsr): # resample 1 channel at a time
   sig, sr = aud
   if (sr == newsr): # Nothing to do
     return aud
   num_channels = sig.shape[0]
   # Resample first channel
   resig = torchaudio.transforms.Resample(sr, newsr)(sig[:1,:])
   if (num_channels > 1):
     # Resample the second channel and merge both channels
     retwo = torchaudio.transforms.Resample(sr, newsr)(sig[1:,:])
     resig = torch.cat([resig, retwo])
   return ((resig, newsr))
```

Resize to the Same Length

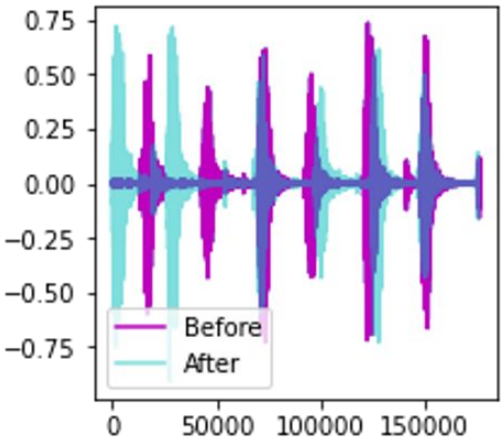
- Two ways: (i) padding short samples with silence, (ii) truncating long samples

```
def pad_trunc(aud, max_ms):
    sig, sr = aud
    num_rows, sig_len = sig.shape
    max_len = sr//1000 * max_ms # // is floor division
    if (sig_len > max_len): # Truncate the signal to the given length
      sig = sig[:,:max_len]
    elif (sig_len < max_len): # padding to add at the beginning and end of the signal
      pad_begin_len = random.randint(0, max_len - sig_len)
      pad_end_len = max_len - sig_len - pad_begin_len
      pad_begin = torch.zeros((num_rows, pad_begin_len)) # Pad with 0s
      pad_end = torch.zeros((num_rows, pad_end_len))
      sig = torch.cat((pad_begin, sig, pad_end), 1)
    return (sig, sr)
```

Data Augmentation: Time Shift

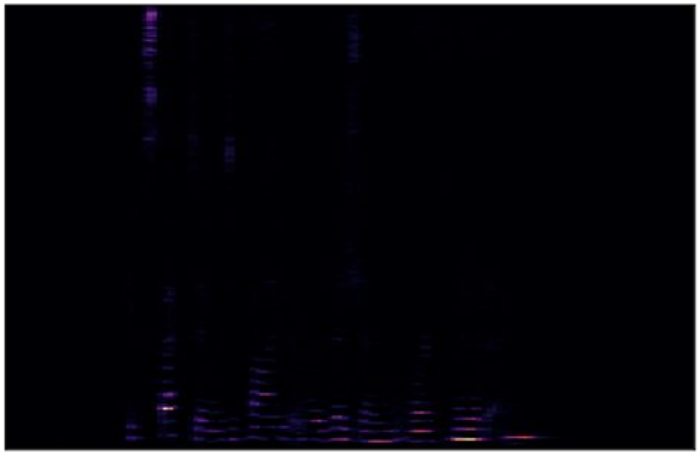
- Signals can be overlapped by a time shift

```
def time_shift(aud, shift_limit):
    sig,sr = aud
    _, sig_len = sig.shape
    shift_amt = int(random.random() * shift_limit * sig_len)
    return (sig.roll(shift_amt), sr)

```

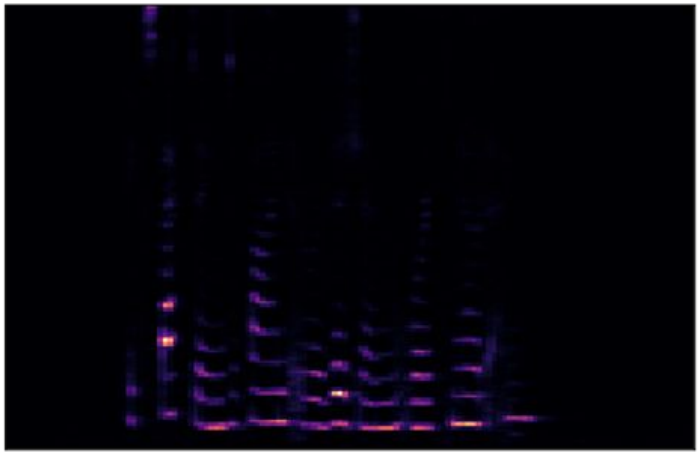


Mel Spectrogram

- Mel spectrogram is a spectrogram that is converted to a Mel scale.
- It uses the Mel Scale instead of Frequency on the y-axis.
- It uses the Decibel Scale instead of Amplitude to indicate colors.



Simple spectrogram

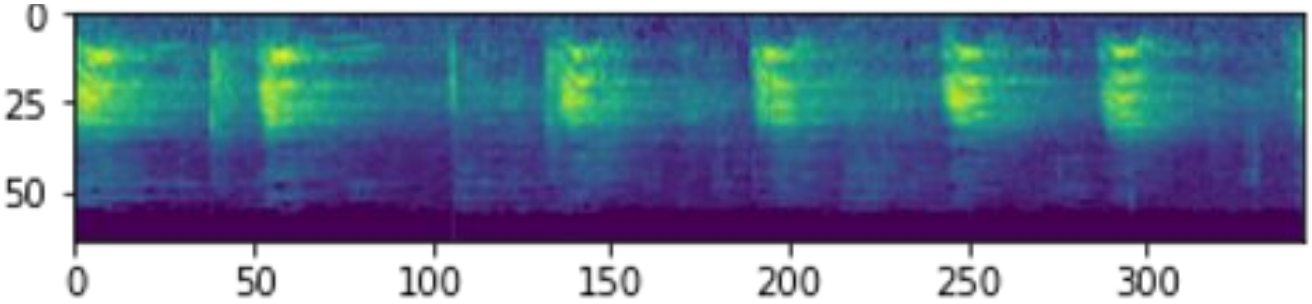


Mel spectrogram

Image credit: https://towardsdatascience.com/audio-deep-learning-made-simple-part-2-why-mel-spectrograms-perform-better-aad889a93505

Mel Spectrogram

```python
def spectro_gram(aud, n_mels=64, n_fft=1024, hop_len=None):
    sig,sr = aud
    top_db = 80
    # spec has shape [channel, n_mels, time], where channel is mono, stereo etc
    spec = transforms.MelSpectrogram(sr, n_fft=n_fft, hop_length=hop_len,
           n_mels=n_mels)(sig)
    # Convert to decibels
    spec = transforms.AmplitudeToDB(top_db=top_db)(spec)
    return (spec)
```

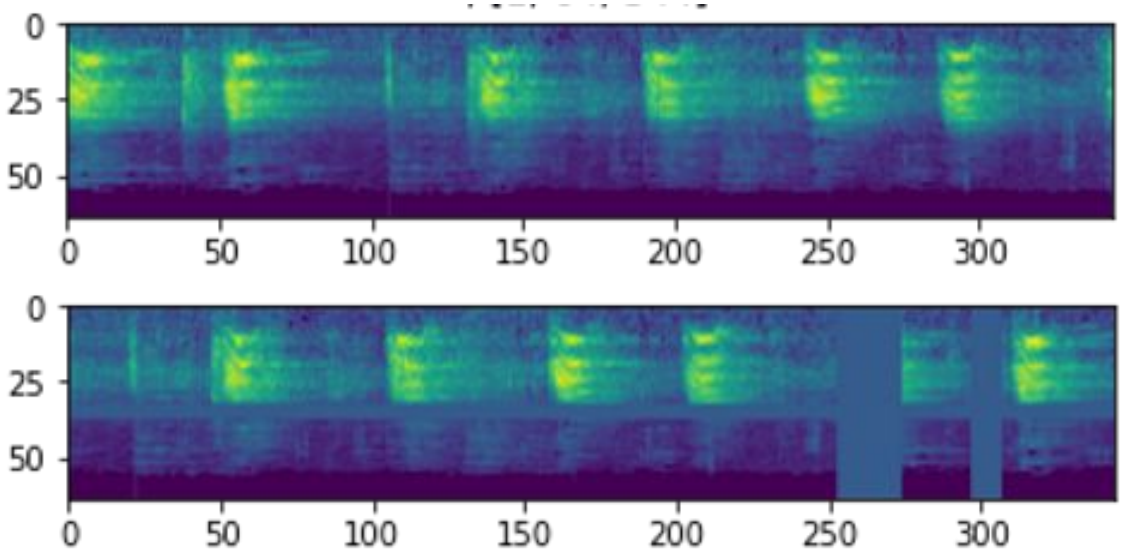


Data Augmentation: Time and Frequency Masking

- **Frequency mask**: randomly mask out a range of consecutive frequencies by adding horizontal bars on the spectrogram.
- **Time mask**: similar to frequency masks, except that we randomly block out ranges of time from the spectrogram by using vertical bars.

Before:

After:



Think about:
How much information (time) do you need if you are asked to guess the name of the song?

Data Augmentation: Time and Frequency Masking

```python
# To prevent overfitting and to help the model generalise better
def spectro_augment(spec, max_mask_pct=0.1, n_freq_masks=1, n_time_masks=1):
    _, n_mels, n_steps = spec.shape
    mask_value = spec.mean()
    aug_spec = spec
    freq_mask_param = max_mask_pct * n_mels
    for _ in range(n_freq_masks):
      aug_spec = transforms.FrequencyMasking(freq_mask_param)(aug_spec, mask_value)
    time_mask_param = max_mask_pct * n_steps
    for _ in range(n_time_masks):
      aug_spec = transforms.TimeMasking(time_mask_param)(aug_spec, mask_value)
    return aug_spec
```

Example: Loading MP3 file to Tensor

- You can simply use torchaudio

```python
import torchaudio

def mp3_to_tensor_torchaudio(mp3_path, target_sr=16000):
    # Load MP3 (requires torchaudio and ffmpeg)
    waveform, sr = torchaudio.load(mp3_path, format="mp3")

    # Resample if needed
    if sr != target_sr:
        resampler = torchaudio.transforms.Resample(sr, target_sr)
        waveform = resampler(waveform)

    return waveform, target_sr

tensor, sr = mp3_to_tensor_torchaudio("YOUR_SONG.mp3")
```

What’s Next?

- We have learnt, using Python, how to process:
  - **Numbers**, **Category**, **Natural Language**, **Images**, **Audios**
- We also show that we can use mathematical methods to transform different types of data into numbers
- We are going to learn how to **apply neural networks to generate embeddings** for all these kind of data